Environment Semantics

Concepts of Programming Languages

CAS CS 320

Outline

- Discuss the difference between **dynamic** and **lexical** scoping
- Introduce **closures** as a way of implementing lexical scoping in the environment model
- Discuss **recursion** and closures

Variable Scoping

Two Major Concerns

Two Major Concerns

1. Are variables *mutable*?

Two Major Concerns

1. Are variables *mutable*?
2. How are variables *scoped*? (Dynamically or lexically?)

Two Major Concerns

1. Are variables *mutable*?

2. How are variables *scoped*? (Dynamically or lexically?)

OCaml variables are immutable, binding defined, and lexically scoped

Mutability

```
let x = 0
let f () =
  let x = 1 in
  ()
print_int x
```

Immutable (OCaml)

```
x = 0
def f():
    global x
    x = 1
print(x)
```

Mutable (Python)

Mutability

```
let x = 0
let f () =
  let x = 1 in
  ()
print_int x
```

Immutable (OCaml)

```
x = 0
def f():
    global x
    x = 1
print(x)
```

Mutable (Python)

Def. *(informal)* A variable is **mutable** if we are allowed to change its value after it has been declared

Scope

Scope

Def. *(Informal)* The **scope** of a variable binding is *when* and *where* a variable can be accessed

Scope

Def. *(Informal)* The **scope** of a variable binding is *when* and *where* a variable can be accessed

There are two standard paradigms: **dynamic** scoping and **lexical** scoping (static scoping)

Scope

Def. *(Informal)* The **scope** of a variable binding is *when* and *where* a variable can be accessed

There are two standard paradigms: **dynamic** scoping and **lexical** scoping (static scoping)

Warning. Scope is one of the most overused terms in computer science...

Dynamic Scoping

```
f() { x=23; g; }
g() { y=$x; }
f
echo $y
```

Bash

Dynamic Scoping

```bash
f() { x=23; g; }
g() { y=$x; }
f
echo $y
```

Bash

Dynamic scoping refers to when bindings are determined at runtime based on *computational context*

Dynamic Scoping

```bash
f() { x=23; g; }
g() { y=$x; }
f
echo $y
```

Bash

Dynamic scoping refers to when bindings are determined at runtime based on *computational context*

This is a *temporal view*, i.e., what a computation done beforehand which affected the value of a variable

Lexical Scoping

```python
x = 0
def f():
    x = 1
    return x
assert(f() == 1)
assert(x == 0)
```

Python

```ocaml
let x = 0
let f () =
  let x = 1 in
  x
let _ = assert (f () = 1)
let _ = assert (x = 0)
```

OCaml

Lexical Scoping

```python
x = 0
def f():
    x = 1
    return x
assert(f() == 1)
assert(x == 0)
```

Python

```ocaml
let x = 0
let f () =
  let x = 1 in
  x
let _ = assert (f () = 1)
let _ = assert (x = 0)
```

OCaml

Lexical (static) scoping refers to the use of textual delimiters to define the scope of a binding

Lexical Scoping

```python
x = 0
def f():
    x = 1
    return x
assert(f() == 1)
assert(x == 0)
```

Python

```ocaml
let x = 0
let f () =
  let x = 1 in
  x
let _ = assert (f () = 1)
let _ = assert (x = 0)
```

OCaml

Lexical (static) scoping refers to the use of textual delimiters to define the scope of a binding

There are two common ways lexical scope is determined:

Lexical Scoping




```python
x = 0
def f():
    x = 1
    return x
assert(f() == 1)
assert(x == 0)
```

Python

```ocaml
let x = 0
let f () =
  let x = 1 in
  x
let _ = assert (f () = 1)
let _ = assert (x = 0)
```

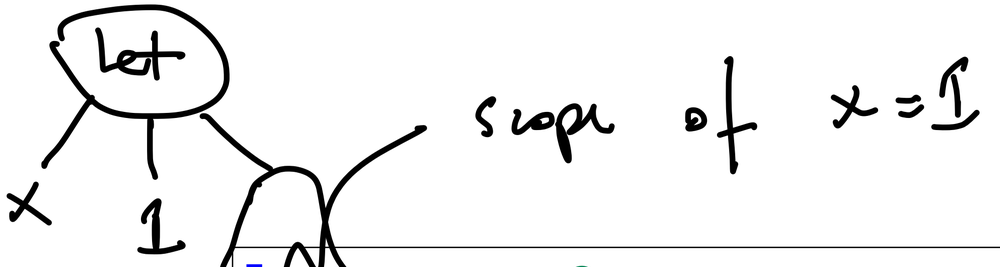
OCaml

Lexical (static) scoping refers to the use of textual delimiters to define the scope of a binding

There are two common ways lexical scope is determined:

- The binding defines it's own scope (let-bindings)

Lexical Scoping




```python
x = 0
def f():
    x = 1
    return x
assert(f() == 1)
assert(x == 0)
```

Python

```ocaml
let x = 0
let f () =
  let x = 1 in
  x
let _ = assert (f () = 1)
let _ = assert (x = 0)
```

OCaml

Lexical (static) scoping refers to the use of textual delimiters to define the scope of a binding

There are two common ways lexical scope is determined:

- The binding defines it's own scope (let-bindings)
- A block defines the scope of a variable (python functions)

Tradeoffs

```
f() { x=23; g; }
g() { y=$x; }
f
echo $y
```

dynamic

vs.

```
let x = 0
let f () =
  let x = 1 in
  x
let _ = assert (f () = 1)
let _ = assert (x = 0)
```

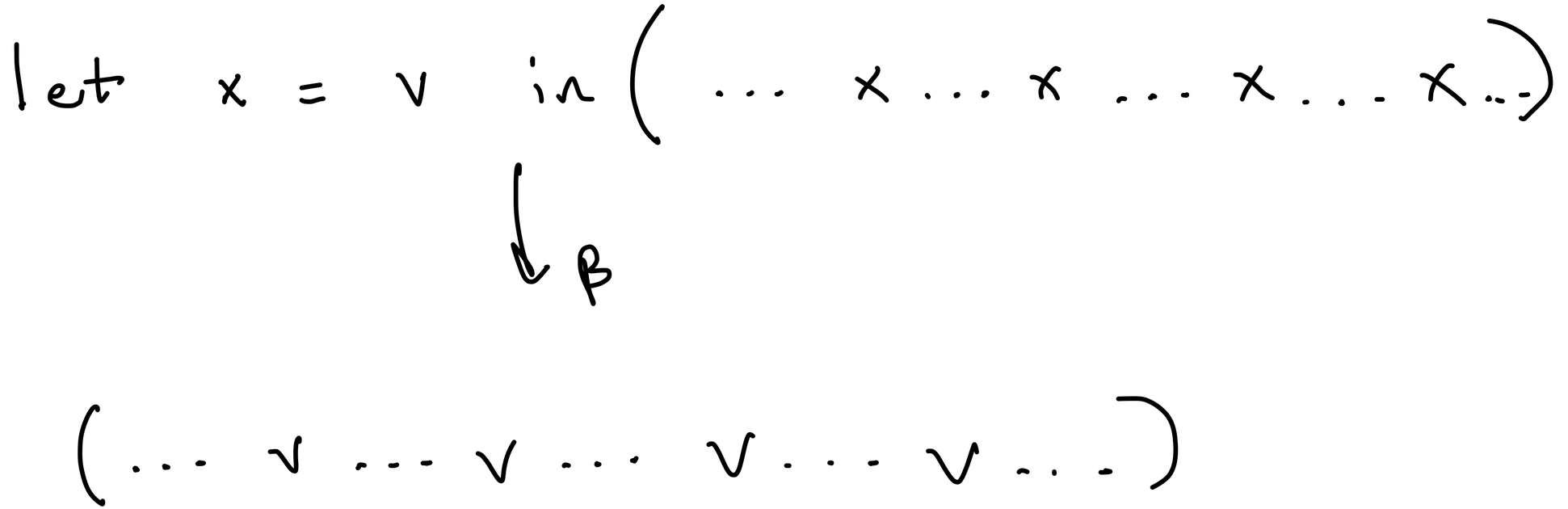
lexical

Implementing dynamic scoping is *way* easier...

But **every modern PL** implements lexical scoping

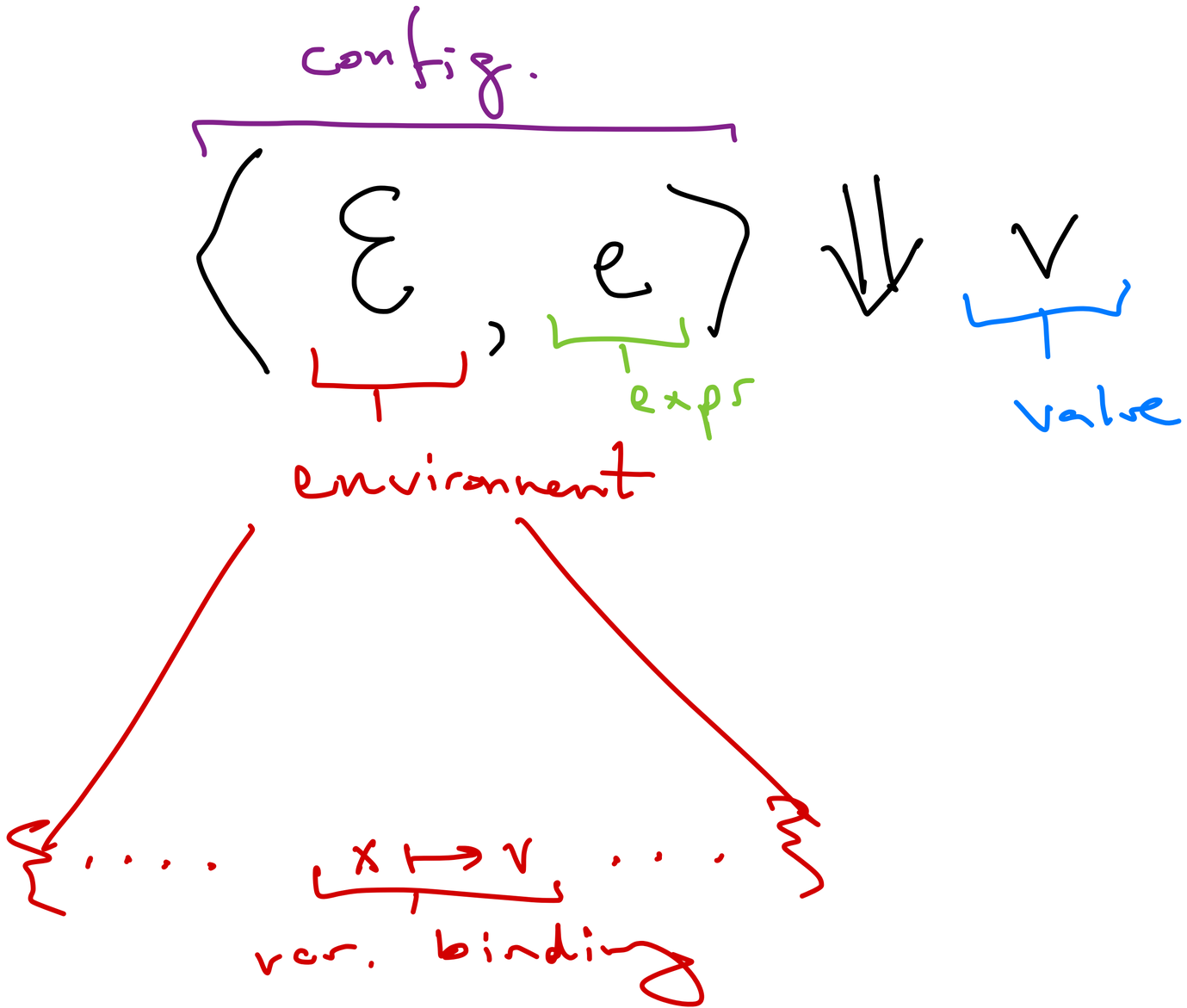
The Environment Model

Motivation



We have to "crawl" whole body of let-expression (inefficient)

Environment Semantic Judgments



Environments

$\mathcal{E}[x \mapsto v](x) = v$

Mapping of variables to values

(Dictionary)

$\{ x \mapsto 0, y \mapsto \text{true}, z \mapsto \lambda y.y \}$

variable binding

shadows

$\mathcal{E}$	env.
$\mathcal{E}[x \mapsto v]$	$\mathcal{E}$ with x bound to v
$\mathcal{E}(x)$	the value x is bound to in $\mathcal{E}$

$(x \mapsto v) \in \mathcal{E} \Rightarrow \mathcal{E}(x) = v$

Syntax (λ Calculus +)

$$e ::= \lambda x.e \mid e\,e \mid x$$

$$\mid \text{let } x = e \text{ in } e \quad \} \text{ let-expressions}$$

$$\mid \text{true} \mid \text{false} \quad \} \text{ Booleans}$$

Environment Semantics (λ Calculus +)

$v ::= \lambda x.e \mid \text{true} \mid \text{false}$

$e ::= \lambda x.e \mid e\,e \mid x$
$\mid \text{let } x = e \text{ in } e$ } let-expressions
$\mid \text{true} \mid \text{false}$ } Booleans

$$\frac{}{\langle \mathcal{E}, \lambda x.e \rangle \Downarrow \lambda x.e}\ (\text{fun})$$

$$\frac{}{\langle \mathcal{E}, x \rangle \Downarrow \mathcal{E}(x)}\ (\text{var})$$

$$\frac{}{\langle \mathcal{E}, \text{true} \rangle \Downarrow \text{true}}\ (\text{true})$$

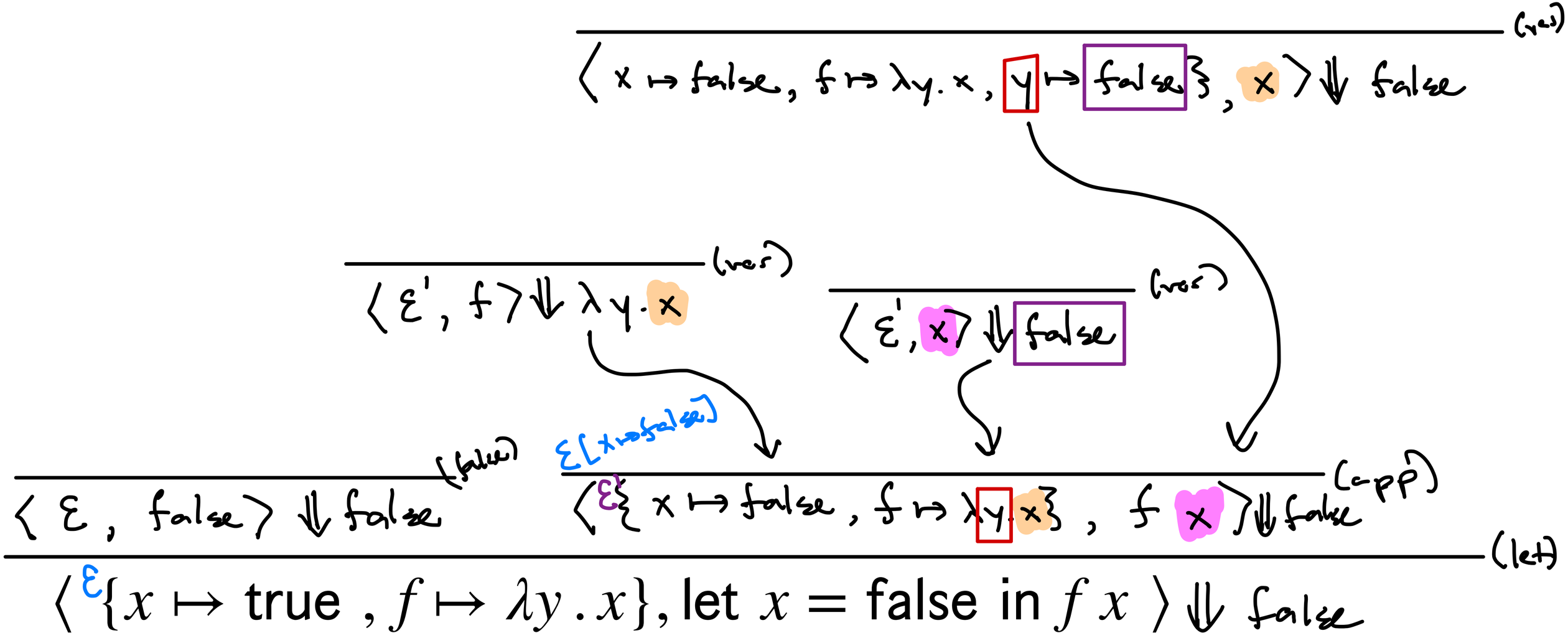
$$\frac{}{\langle \mathcal{E}, \text{false} \rangle \Downarrow \text{false}}\ (\text{false})$$

subst. sem:
$[v_2/x]e \Downarrow v$

$$\frac{\langle \mathcal{E}, e_1 \rangle \Downarrow \lambda x.e \qquad \langle \mathcal{E}, e_2 \rangle \Downarrow v_2 \qquad \langle \mathcal{E}[x \mapsto v_2], e \rangle \Downarrow v}{\langle \mathcal{E}, e_1\, e_2 \rangle}\ (\text{app})$$

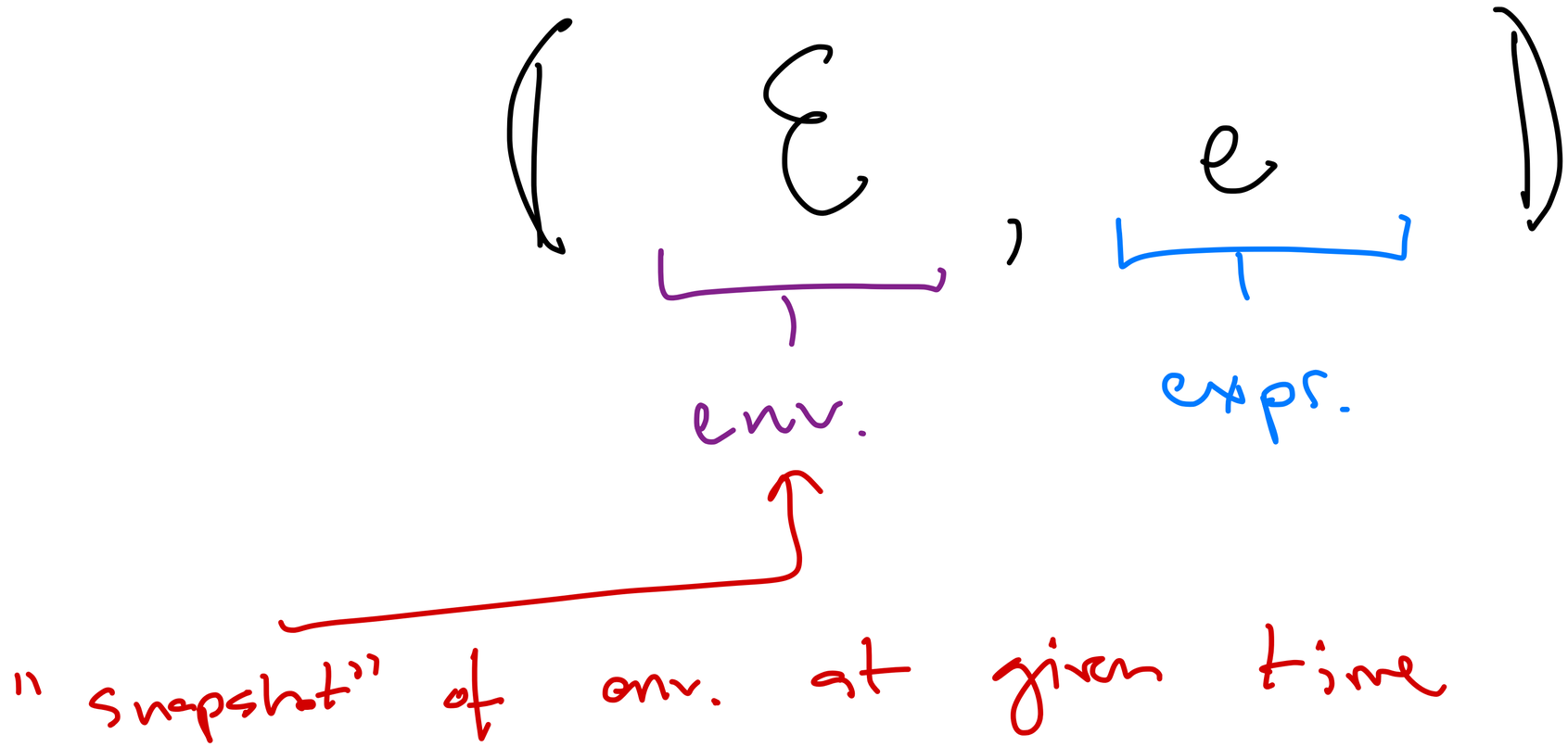
$$\frac{\langle \mathcal{E}, e_1 \rangle \Downarrow v_1 \qquad \langle \mathcal{E}[x \mapsto v_1], e_2 \rangle \Downarrow v_2}{\langle \mathcal{E}, \text{let } x = e_1 \text{ in } e_2 \rangle \Downarrow v_2}\ (\text{let})$$

Back to Scoping



Closures

Closures



Values (λ Calculus +)

$$v ::= \underbrace{(\!| \mathcal{E}, \lambda x.e |\!)}_{\text{fun. val.}} \mid \text{true} \mid \text{false}$$

Semantics Again (λ Calculus +)

$$v ::= \langle\!| \mathcal{E}, \lambda x.e |\!\rangle \mid \text{true} \mid \text{false}$$

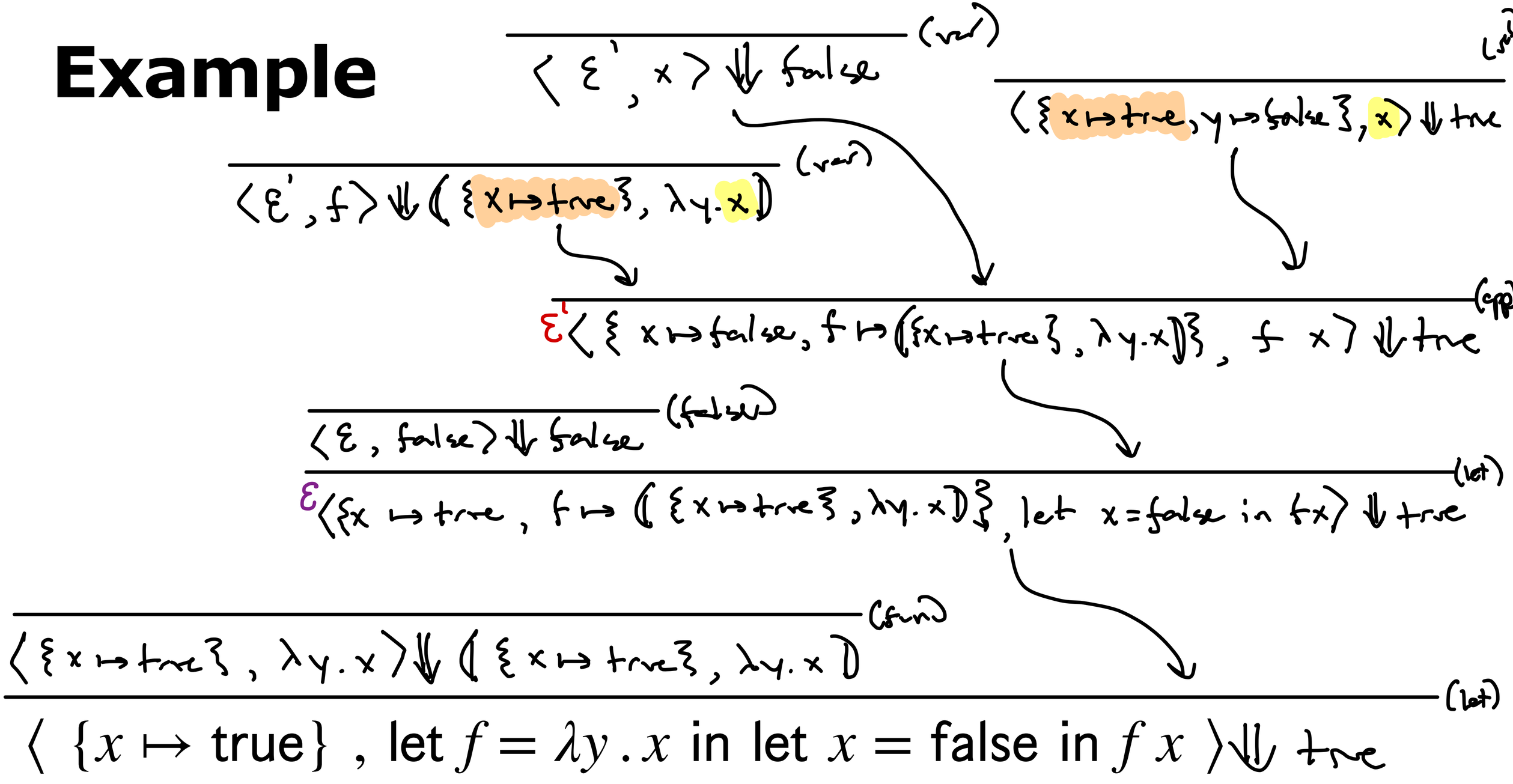
fun. val.

$$e ::= \lambda x.e \mid e\,e \mid x$$
$$\mid \text{let } x = e \text{ in } e \quad \} \text{ let-expressions}$$
$$\mid \text{true} \mid \text{false} \quad \} \text{ Booleans}$$

$$\frac{}{\langle \mathcal{E}, \lambda x.e \rangle \Downarrow \langle\!| \mathcal{E}, \lambda x.e |\!\rangle} \text{(fun)}$$

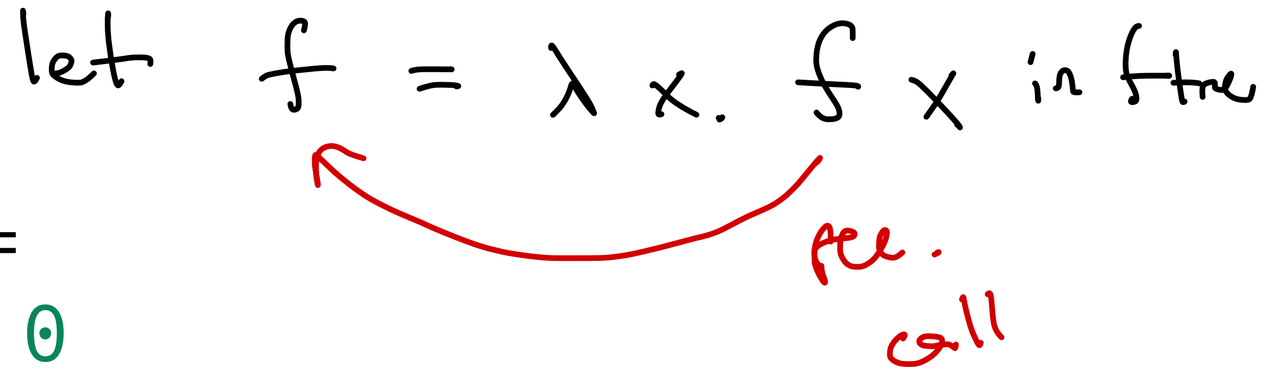
$$\frac{\langle \mathcal{E}, e_1 \rangle \Downarrow \langle\!| \mathcal{E}', \lambda x.e |\!\rangle \quad \langle \mathcal{E}, e_2 \rangle \Downarrow v_2 \quad \langle \mathcal{E}'[x \mapsto v_2], e \rangle \Downarrow v}{\langle \mathcal{E}, e_1\, e_2 \rangle \Downarrow v} \text{(app)}$$

Example


⟨ ε' , x ⟩ ⇓ false (var)
⟨ ε' , f ⟩ ⇓ ({ x ↦ true } , λy.x) (var)
⟨ { x ↦ true , y ↦ false } , x ⟩ ⇓ true (var)
ε' ⟨ { x ↦ false , f ↦ ({ x ↦ true } , λy.x) } , f x ⟩ ⇓ true (app)
⟨ ε , false ⟩ ⇓ false (false)
ε ⟨ { x ↦ true , f ↦ ({ x ↦ true } , λy.x) } , let x = false in f x ⟩ ⇓ true (let)
⟨ { x ↦ true } , λy.x ⟩ ⇓ ({ x ↦ true } , λy.x) (fun)
⟨ { x ↦ true } , let f = λy.x in let x = false in f x ⟩ ⇓ true (let)

Recursion

High-Level

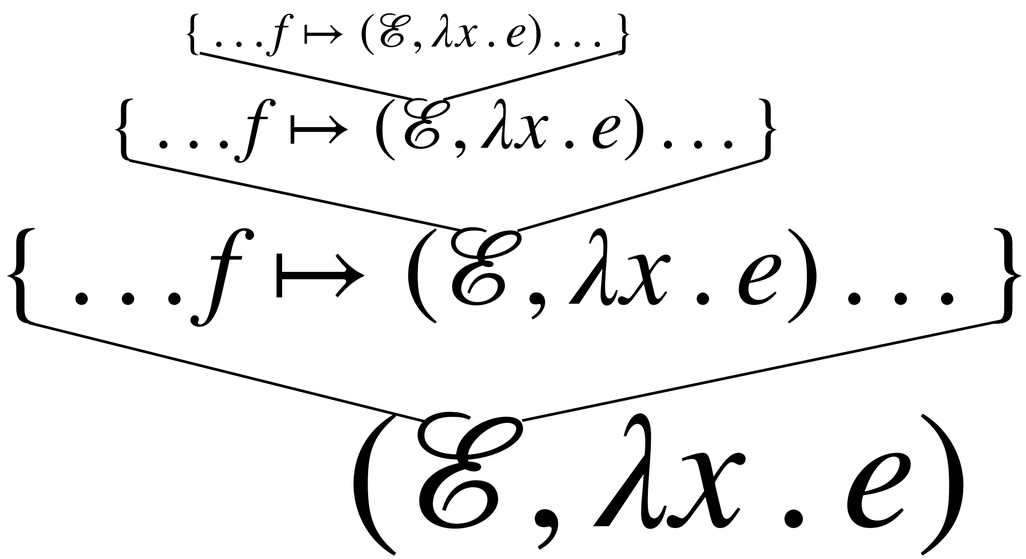



```
let f x =
  if x = 0
  then 1
  else f (x - 1)
in f 10
```

We've only considered *non-recursive* function so far

And there's no easy way to do recursion with substitution

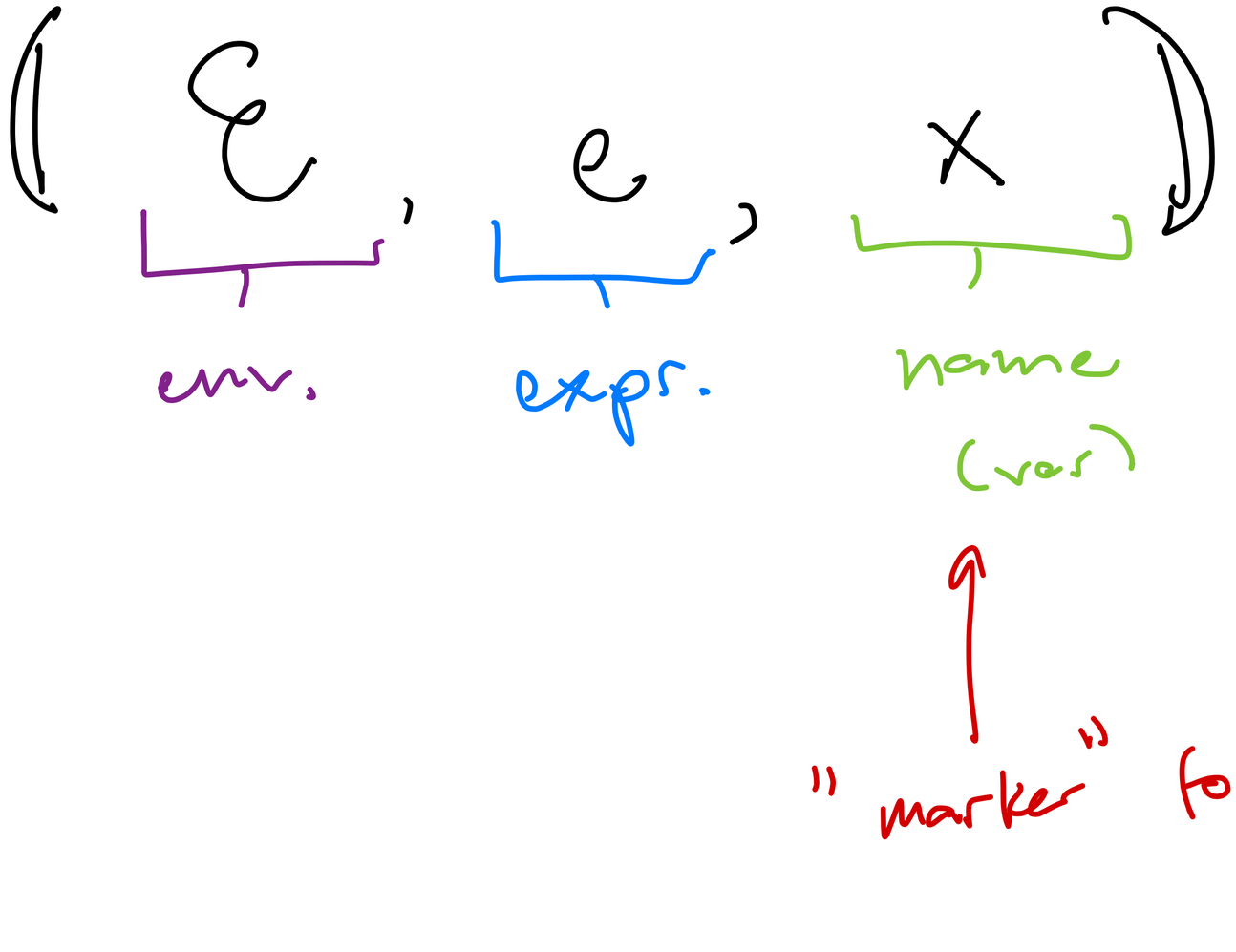
The Problem



In order to implement recursion, a closure has to "*know thyself*"

But we **can't** build infinite structures like this in OCaml

Named Closures



Lambda Calculus^{++} (Syntax, Again)

Semantics Again Again (λ Calculus +)

$$v ::= \langle\!| \mathcal{E}, \lambda x.e |\!\rangle \mid \langle\!| \mathcal{E}, \lambda x.e, x |\!\rangle \mid \text{true} \mid \text{false}$$

$$e ::= \lambda x.e \mid e\,e \mid x \mid \underbrace{\text{let } x = e \text{ in } e}_{\text{let-expressions}} \mid \underbrace{\text{true} \mid \text{false}}_{\text{Booleans}}$$

$$\frac{\langle \mathcal{E}, e_1 \rangle \Downarrow \langle\!| \mathcal{E}', \lambda x.e, f |\!\rangle \quad \langle \mathcal{E}, e_2 \rangle \Downarrow v_2 \quad \langle \mathcal{E}'[f \mapsto \langle\!| \mathcal{E}', \lambda x.e, f |\!\rangle][x \mapsto v_2], e \rangle \Downarrow v}{\langle \mathcal{E}, e_1\, e_2 \rangle \Downarrow v} \text{(appRec)}$$

$$\frac{\langle \mathcal{E}[x \mapsto \langle\!| \mathcal{E}, \lambda x_1.e_1, x |\!\rangle], e_2 \rangle \Downarrow v_2}{\langle \mathcal{E}, \text{let } x = \lambda x_1.e_1 \text{ in } e_2 \rangle \Downarrow v_2} \text{(letRec)}$$

demo